```
#include <stdio.h> #include <stdbool.h> #include <SDL2/SDL.h>

#define TITLE           "game" #define WINDOW_WIDTH     800 #define WINDOW_HEIGHT    600 #define FPS
60

#define PLAYER_SPEED     4.0f #define SPRINT_MULTIPLIER 2.0f

#define PLAYER_WIDTH     48 #define PLAYER_HEIGHT    48

SDL_Window *window = NULL; SDL_Renderer *renderer = NULL;


struct player {

  float x, y;
  bool moveLeft, moveRight, moveUp, moveDown;
  bool sprinting;
  int facing;

} player;


int shouldQuit = 0;


void initSDL(){

  SDL_Init(SDL_INIT_EVERYTHING);

  window = SDL_CreateWindow(
    TITLE,
    0, //SDL_WINDOWPOS_CENTERED,
    0, //SDL_WINDOWPOS_CENTERED,
    WINDOW_WIDTH,
    WINDOW_HEIGHT,
    SDL_WINDOW_SHOWN
  );

  renderer = SDL_CreateRenderer(window, -1, 0);

}

void cleanup(){

  SDL_DestroyRenderer(renderer);
  SDL_DestroyWindow(window);
  SDL_Quit();

}

void setupLevel(){
  // TODO: add tilemap?

  player.x = (WINDOW_WIDTH / 2) - PLAYER_WIDTH;
  player.y = (WINDOW_HEIGHT / 2) - PLAYER_HEIGHT;
  player.vx = 0;
  player.vy = 0;

}


void processInput(){

  SDL_Event event;
  while (SDL_PollEvent(&event)){
    switch (event.type) {
```

```
      case SDL_QUIT:
        shouldQuit = 1;
        break;
      case SDL_KEYUP:
        if (event.key.keysym.sym == SDLK_ESCAPE)
          shouldQuit = 1;
        if (event.key.keysym.sym == SDLK_q)
          shouldQuit = 1;
        break;
    }
  }
  const Uint8 *state = SDL_GetKeyboardState(NULL);

  player.moveUp = state[SDL_SCANCODE_W];
  player.moveDown = state[SDL_SCANCODE_S];
  player.moveLeft = state[SDL_SCANCODE_A];
  player.moveRight = state[SDL_SCANCODE_D];
  player.sprinting = state[SDL_SCANCODE_LSHIFT]; }


void update(){
  // TODO: collision detection, on other objects
  // TODO: collision on tilemap?
  // TODO: debug text with player coords


  //player movement


  player.vx = 0;
  player.vy = 0;

  if (player.moveUp) {
    player.facing = 1;
    if (player.sprinting) {
      player.y -= PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
      player.y -= PLAYER_SPEED;
  }
  if (player.moveDown) {
    player.facing = 2;
    if (player.sprinting) {
      player.y += PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
      player.y += PLAYER_SPEED;
  }
  if (player.moveLeft) {
    player.facing = 3;
    if (player.sprinting) {
      player.x -= PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
      player.x -= PLAYER_SPEED;
  }
  if (player.moveRight) {
    player.facing = 4;
    if (player.sprinting) {
      player.x += PLAYER_SPEED * SPRINT_MULTIPLIER;
    } else
      player.x += PLAYER_SPEED;
  }


  //border collision


  if (player.y <= 0) {
```

```
    player.y = 0;
  }
  if (player.y >= WINDOW_HEIGHT - PLAYER_HEIGHT) {
    player.y = WINDOW_HEIGHT - PLAYER_HEIGHT;
  }
  if (player.x <= 0) {
    player.x = 0;
  }
  if (player.x >= WINDOW_WIDTH - PLAYER_WIDTH) {
    player.x = WINDOW_WIDTH - PLAYER_WIDTH;
  }

  SDL_Delay(1000 / FPS); }

void render(){
  // TODO: render player sprite
  // TODO: render tile map
  // TODO: render debug text

  SDL_SetRenderDrawColor(renderer, 0, 0, 0, 255);
  SDL_RenderClear(renderer);

  SDL_Rect player_rect = {
    player.x,
    player.y,
    PLAYER_WIDTH,
    PLAYER_HEIGHT
  };

  SDL_SetRenderDrawColor(renderer, 255, 255, 255, 255);
  SDL_RenderFillRect(renderer, &player_rect);

  SDL_RenderPresent(renderer);

}

int main(){

  initSDL();
  setupLevel();

  while (shouldQuit != 1){

    processInput();
    update();
    render();

  }

  cleanup();
  printf("All good!0);
  return 0; }
```